

SE373 · KỸ THUẬT XÂY DỰNG HỆ THỐNG AGENTIC AI

BÁO CÁO BÀI TẬP VỀ NHÀ 03

DỰNG AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN VÀ LANGGRAPH

Lớp harness · ReAct · Plan-then-Execute · Lai · Đánh giá

Giảng viên hướng dẫn Nguyễn Hiếu Nghĩa

Sinh viên thực hiện Nguyễn Trần Đăng Khoa - MSSV 24520834

Mã nguồn <https://github.com/Khoa24520834/SE373-BTVN03>

MỤC LỤC

- 1. Giới thiệu..... 3
 - 1.1. Bài toán 3
 - 1.2. Yêu cầu của đề và cách đáp ứng 3
 - 1.3. Công nghệ sử dụng 3
- 2. Kiến trúc tổng thể..... 4
- 3. Tool mockup..... 4
 - 3.1. Thế giới giả tăt định 4
 - 3.2. Năm tool 5
 - 3.3. Observation có cấu trúc 5
 - 3.4. Dữ liệu và các “mồi nhử” ràng buộc 5
 - 3.5. Bốn kịch bản đánh giá..... 5
- 4. Lớp harness 6
 - 4.1. Bốn lớp theo yêu cầu đề bài 6
 - 4.2. Ràng buộc là dữ liệu 6
 - 4.3. Tiêu chí hoàn thành kiểm bằng code 6
 - 4.4. Kiểm quyền 7
 - 4.5. Bàn giao cho con người 7
 - 4.6. Các lớp phụ trợ 8
 - 4.7. Thứ tự kiểm trong mỗi vòng..... 8
- 5. Ba mẫu thiết kế..... 8
 - 5.1. ReAct 8
 - 5.2. Plan-then-Execute..... 9
 - 5.3. Lai (Plan + replan) 10
 - 5.4. So sánh cách cài đặt..... 10
- 6. Kiểm thử..... 10
- 7. Đánh giá 11
 - 7.1. Phương pháp 11
 - 7.2. Kết quả với model giả 11
 - 7.3. Nhận xét..... 12
 - 7.4. Thử nghiệm với Gemini 3.8 Flash 13
- 8. So sánh và thảo luận 14
- 9. Hạn chế và hướng phát triển 14
- 10. Kết luận 14
- Phụ lục A. Hướng dẫn chạy 15
- Phụ lục B. Mã nguồn 15

1. Giới thiệu

1.1. Bài toán

Agent nhận một yêu cầu bằng ngôn ngữ tự nhiên: *đặt một vé bay từ TP.HCM (SGN) đi Đà Nẵng (DAD) ngày 07/10/2026, khởi hành trước 12:00, giá không quá 2.000.000đ, chọn chuyến rẻ nhất thoả yêu cầu*. Để đạt mục tiêu, agent phải gọi lần lượt các tool tìm chuyến, kiểm tra ghế, giữ chỗ và thanh toán. Hai bước cuối là hành động có **tác dụng phụ không hoàn tác**, nên sai một lần là có thể mất tiền thật.

Theo bài giảng, **agent = goal + tools + loop + termination**, và trong năm bước của một vòng lặp chỉ có bước “model đề xuất tool” thuộc về model; dựng ngữ cảnh, gọi tool, ghi kết quả và xét điều kiện dừng là code của **harness**. Bài làm hiện thực hoá đúng ranh giới này: model chỉ đề xuất, harness quyết định.

1.2. Yêu cầu của đề và cách đáp ứng

Yêu cầu	Cách đáp ứng	Mục
Tìm hiểu LangChain, LangGraph; tạo tool mockup	create_agent + middleware (LangChain 1.x); StateGraph, interrupt(), checkpoint (LangGraph); 5 tool mockup tất định	3, 5
01 · Đủ các lớp: ràng buộc là dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền, bàn giao	harness.py: Constraints, is_done(), check_permission(), Handoff; kèm phát hiện lặp, ngân sách, chống bịa	4
02 · Cài đặt agent với 3 mẫu: ReAct, Plan-then-Execute, Lai	agent_react.py, agent_plan.py, agent_hybrid.py, dùng chung một harness	5
03 · Đánh giá hiệu quả của 3 mẫu	evaluate.py: 3 mẫu × 4 kịch bản, với model giả và Gemini 3.8 Flash	7, 8

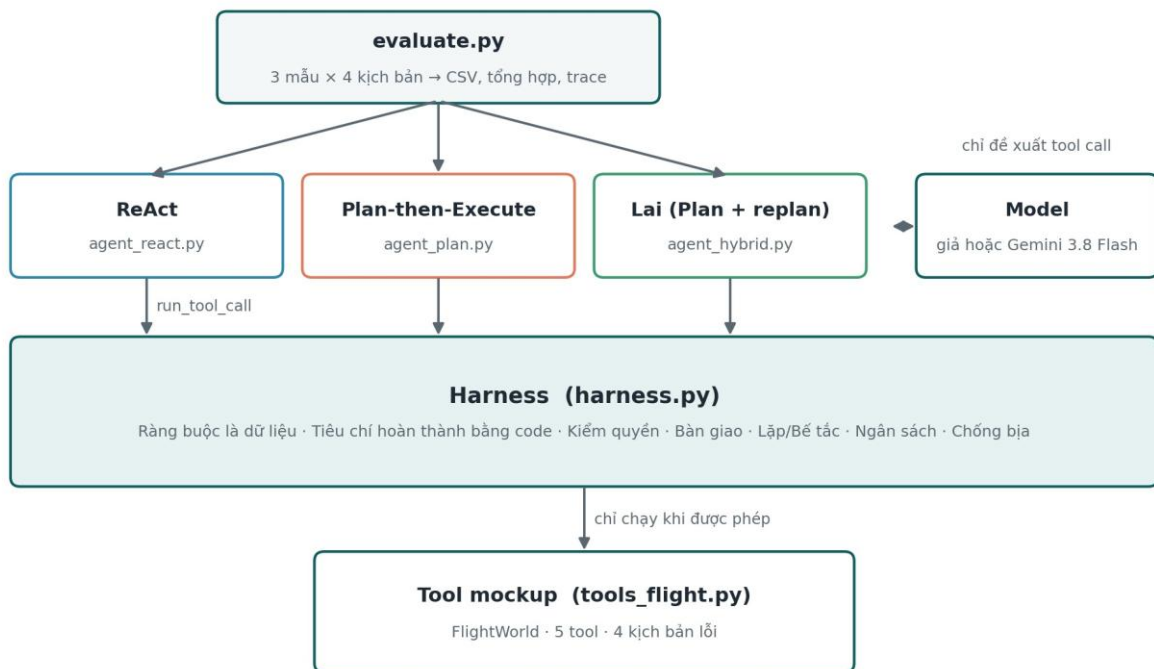
1.3. Công nghệ sử dụng

Thành phần	Phiên bản	Vai trò
Python	3.14	Ngôn ngữ cài đặt
LangChain	1.4.3	create_agent, middleware, tool calling
LangGraph	1.2.12	StateGraph, interrupt(), InMemorySaver
Pydantic	2.13.5	Schema Booking, Plan
pytest	9.1.1	190 test tự động, chạy offline
langchain-google-genai	4.4.0	Gọi model thật Gemini 3.8 Flash

Nguyên tắc thiết kế xuyên suốt

Model chỉ đề xuất lời gọi tool. Mọi quyết định có hậu quả (tool có được chạy không, đã xong chưa, có phải dừng không, bàn giao gì cho người) đều do code của harness đưa ra. Harness dùng chung cho cả ba mẫu, nên khác biệt khi đánh giá chỉ đến từ cách điều phối model, và phép so sánh là công bằng.

2. Kiến trúc tổng thể



Hình 1. Kiến trúc tổng thể: ba mẫu dùng chung một harness và một bộ tool mockup

Mỗi mẫu thiết kế chỉ khác nhau ở cách điều phối model. Mọi lời gọi tool, dù đến từ mẫu nào, đều đi qua `Harness.run_tool_call`; tool chỉ được thực thi khi harness cho phép. Model có thể là **model giả** (tắt định, miễn phí, dùng để đo cơ chế và để tái hiện lỗi) hoặc **Gemini 3.8 Flash** (đọc cấu hình từ file `.env`). `evaluate.py` chạy mọi tổ hợp và ghi kết quả. Mã nguồn gồm khoảng 2.900 dòng Python và 190 test.

File	Vai trò
<code>tools_flight.py</code>	Tool mockup, dữ liệu chuyến bay cố định, 4 kịch bản lỗi
<code>harness.py</code>	Các lớp harness: ràng buộc, tiêu chí hoàn thành, kiểm quyền, bàn giao, lập, ngân sách
<code>common.py</code>	Model giả, đọc file <code>.env</code> , dựng model thật, định dạng trace
<code>agent_react.py</code>	Mẫu 1: ReAct
<code>agent_plan.py</code>	Mẫu 2: Plan-then-Execute (kèm cơ chế lập lại kế hoạch)
<code>agent_hybrid.py</code>	Mẫu 3: Lai
<code>evaluate.py</code>	Chạy 3 mẫu × các kịch bản, ghi CSV, bảng tổng hợp và trace
<code>check_api.py</code>	Kiểm tra API key, model và khả năng gọi tool trước khi chạy thật

3. Tool mockup

3.1. Thế giới giả tắt định

FlightWorld giữ toàn bộ trạng thái của **một** lần chạy: dữ liệu chuyến, ghế đã đặt, các đặt chỗ và nhật ký gọi tool. Mỗi lần chạy tạo một world mới nên không rõ trạng thái giữa các lần; cùng kịch bản và cùng chuỗi lời

gọi luôn cho cùng kết quả. Nhờ vậy lỗi tái hiện được (bước “Tái hiện” trong quy trình debug của bài giảng) và so sánh giữa các mẫu là công bằng. world.trace ghi mọi lời gọi; world.side_effects lọc ra các lời gọi book_seat/pay thành công, phục vụ bàn giao.

3.2. 5 tool

Tool	Tham số	Kết quả	Tác dụng phụ
search_flights	origin, dest, date (YYYY-MM-DD)	Danh sách chuyến: mã, giờ, giá, hoàn được	Không
check_seat	flight	Số ghế còn, giá, hoàn được	Không
book_seat	flight, seat	Đặt chỗ ở trạng thái held	Có
pay	code, method	Đặt chỗ confirmed, đã trả tiền	Có, không hoàn tác
get_booking	code	Trạng thái đặt chỗ	Không

3.3. Observation có cấu trúc

Mọi tool trả JSON có trường status: ok, invalid_param, not_found, sold_out, seat_taken, already_paid hoặc error; mọi lỗi kèm hint chỉ hướng đi khác. Thiết kế này xử lý trực tiếp hai lỗi được nêu trong bài giảng:

- **Tin vào dữ liệu sai:** “không có chuyến” ({"status": "ok", "flights": []}) được tách khỏi “công cụ lỗi” ({"status": "error", ...}), nên agent không biến một lỗi thành khẳng định.
- **Lặp không tiến bộ:** lỗi tham số trả gợi ý cụ thể, ví dụ “Dùng YYYY-MM-DD, ví dụ 2026-10-07”, thay cho thông báo chung chung “not found” khiến agent thử lại mù quáng.

3.4. Dữ liệu và các “môi nhử” ràng buộc

Dữ liệu gồm 12 chuyến cố định. Ngoài ba chuyến hợp lệ, mỗi ràng buộc đều có chuyến “môi nhử” để kiểm tra agent và harness có tuân thủ ràng buộc hay không.

Nhóm	Chuyến	Kiểm tra điều gì
Thoả mọi ràng buộc	VJ604 (06:30, 1.290.000đ, hoàn được) · VJ606 (07:20, 1.420.000đ) · VN122 (08:10, 1.850.000đ, không hoàn)	Ứng viên hợp lệ; chọn rẻ nhất
Buổi sáng nhưng vượt giá	VJ612 (2.080.000đ) · QH120 (2.450.000đ)	Ràng buộc giá
Rẻ nhưng sau 12:00	BL6180 (12:30) · QH118 (15:40) · VJ608 (20:15), cùng hai chuyến chiều giá cao	Ràng buộc giờ
Sai ngày hoặc sai điểm đến	VJ610 (08/10) · VN126 (SGN → HUI)	Ràng buộc ngày, tuyến

3.5. Bốn kịch bản đánh giá

Mã	Kịch bản	Cơ chế	Kết thúc lý tưởng
S1	happy	Không chèn lỗi. Chuyến rẻ nhất thoả ràng buộc là VJ604, hoàn được và dưới hạn mức	A · đạt mục tiêu
S2	timeout	check_seat luôn trả lỗi timeout	C · phát hiện lặp (hoặc A nếu agent tự tìm đường vòng)

S4	approval	VJ604 và VJ606 không còn; chỉ VN122 (không hoàn, vượt hạn mức 1.500.000đ) thoả ràng buộc	E · dừng chờ người duyệt, chưa có tác dụng phụ
S6	env_change	Sau lần tìm kiếm đầu tiên, VJ604 hết ghế: kế hoạch lập sẵn bị lỗi thời	A · thích nghi sang VJ606

Mã kịch bản giữ theo kế hoạch ban đầu gồm sáu kịch bản. Hai kịch bản còn lại (S3: mọi chuyến vượt giá; S5: model bịa kết quả) được kiểm bằng test tự động thay vì đưa vào đợt đánh giá.

4. Lớp harness

Toàn bộ harness nằm trong harness.py, độc lập với framework, nên cả ba mẫu dùng chung. Bảng dưới ánh xạ bốn lớp đề bài yêu cầu tới phần cài đặt và tới lỗi tương ứng trong bài giảng.

4.1. Bốn lớp theo yêu cầu đề bài

Lớp	Cài đặt	Lỗi được chặn
Ràng buộc là dữ liệu	Constraints: tuyến, ngày, giờ trước 12:00, giá ≤ 2.000.000đ; kiểm lại trước book_seat, pay và khi xét hoàn thành	Quên yêu cầu ban đầu
Tiêu chí hoàn thành bằng code	Harness.is_done() đọc lại kho đặt chỗ, kiểm 4 nhóm điều kiện	Model tự tuyên bố xong; bịa đặt
Kiểm quyền	check_permission() và Policy (hạn mức 1.500.000đ, vé không hoàn, tối đa 1 vé), chạy TRƯỚC khi thực thi	Tác dụng phụ vượt thẩm quyền
Bàn giao	Handoff: trạng thái, tác dụng phụ đã xảy ra, hướng đã thử, câu hỏi cụ thể	Dừng bất thường mà im lặng

4.2. Ràng buộc là dữ liệu

<pre>@dataclass(frozen=True) class Constraints: origin: str = "SGN" dest: str = "DAD" date: str = "2026-10-07" depart_before: str = "12:00" max_price: int = 2_000_000</pre>

Vì là dữ liệu bất biến đặt ở một chỗ, yêu cầu không “lùi xa dần” khi nhật ký dài ra (nguyên nhân của lỗi “quên yêu cầu” trong bài giảng). Cùng một đối tượng được dùng để: sinh câu yêu cầu và đoạn nhắc ràng buộc cho model (request_text(), to_prompt()); từ chối book_seat/pay với chuyến vi phạm; kiểm kế hoạch của Plan-then-Execute (lint_plan); và xét tiêu chí hoàn thành.

4.3. Tiêu chí hoàn thành kiểm bằng code

is_done() đọc thẳng kho đặt chỗ của world (nguồn sự thật) thay vì gọi tool, để không làm bản trace. Tác vụ chỉ được coi là xong khi đồng thời:

- có **đúng một** đặt chỗ đã thanh toán, ở trạng thái confirmed;
- đặt chỗ đó thoả cả bốn ràng buộc;
- giá trong đặt chỗ khớp giá đã thấy ở check_seat (kiểm chứng chéo);
- mọi observation chứa đặt chỗ đều parse được theo schema Booking (Pydantic).

Dạng tiêu chí kiểm chứng được (bài giảng)	Cài đặt
---	---------

Vị từ chạy bằng code	is_done(): confirmed, đã trả tiền, giá, ngày, giờ, tuyến, đúng một vé
Schema hợp lệ	Booking.model_validate cho mọi observation có đặt chỗ
Kiểm chứng chéo	Đọc lại kho đặt chỗ; so giá với kết quả check_seat
Người duyệt	Kiểm quyền (mục 4.4) và duyệt kế hoạch (mục 5.2)

Hệ quả quan trọng

Khi model ngừng gọi tool và tuyên bố đã xong, verify_final() chỉ chấp nhận nếu is_done() đúng; nếu không, harness phản hồi lý do và cho model làm tiếp, sai lần thứ hai thì dừng. Khi đạt mục tiêu, câu trả lời cuối do harness dựng từ dữ liệu đã kiểm chứng (summary()), nên không thể bịa. Mục 7.4 cho thấy lớp này bắt được một trường hợp thật với Gemini.

4.4. Kiểm quyền

Chỉ hai tool có tác dụng phụ (book_seat, pay) cần kiểm. Harness từ chối vì sai dữ liệu hoặc sai ràng buộc trước, rồi mới hỏi người, vì hỏi người về một chuyến đã sai ràng buộc là vô nghĩa.

Tình huống	Quyết định	Ví dụ
Tool không có tác dụng phụ	ALLOW	search_flights, check_seat, get_booking
Chuyến chưa xuất hiện trong kết quả tìm kiếm	DENY · unknown_flight	Mã chuyến bịa VN999
Chuyến vi phạm ràng buộc	DENY · violates_constraints	VJ612, giá 2.080.000đ
Đã có một đặt chỗ	DENY · already_booked	Đặt thêm vé thứ hai
Giá trên 1.500.000đ hoặc vé không hoàn, chưa được duyệt	NEED_HUMAN → dừng E	VN122, 1.850.000đ, không hoàn

DENY không chạy tool mà trả observation kèm gợi ý để model đổi hướng. NEED_HUMAN không chạy tool, dừng vòng lặp và lập bản bàn giao.

4.5. Bàn giao cho con người

Bản bàn giao thật ở kịch bản S4, ngay khi model định giữ chỗ VN122:

[Kết E · Cần con người] dừng bình thường
Trạng thái : đã thấy 8 chuyến, 1 chuyến thoả mọi ràng buộc
(rẻ nhất: VN122 1.850.000đ); chưa giữ chỗ
Tác dụng phụ : chưa có (chưa giữ chỗ, chưa trả tiền)
Đã thử : không có lỗi nào
Đang chờ duyệt : book_seat(flight='VN122', seat='12A') · 1.850.000đ
· vé không hoàn · vượt hạn mức 1.500.000đ Câu hỏi : Duyệt đặt VN122 ghế 12A (1.850.000đ, vé không hoàn) không?

Dòng “Tác dụng phụ: chưa có” chứng minh kiểm quyền chạy trước khi thực thi. Bản bàn giao trả lời đúng ba điều bài giảng yêu cầu (đã làm tới đâu, đã thử gì, cần hỏi gì) để người nhận trả lời được trong 30 giây.

4.6. Các lớp phụ trợ

- **Phát hiện lặp và bế tắc** (LoopDetector): ba tín hiệu của bài giảng là trùng (tool, tham số) trong cửa sổ 6 vòng, trùng observation lỗi dù tham số khác, và đại lượng tiến triển đứng yên 5 vòng. Đại lượng tiến triển là $10 \times$ số ràng buộc được thỏa tốt nhất + giai đoạn đặt chỗ. So với code mẫu trong slide có hai điều chỉnh: báo động ở lần trùng **thứ ba** (khớp ví dụ “V2, V3, V4 timeout, V4 báo động”), và riêng get_booking dùng cả observation làm dấu vân tay để việc chờ trạng thái chuyển từ held sang confirmed không bị báo nhầm.
- **Ngân sách**: 12 lần gọi model, 20 lần gọi tool, 40.000 token, thời gian (120 giây với model giả, 600 giây với model thật) và 0,5 USD mỗi lượt.
- **Kiểm tra lời gọi** (validate_tool_call): chỉ cho phép tên tool trong danh sách, đủ và đúng tham số, đúng định dạng ngày và ghế.
- **Chống bịa** (grounding_check): mã chuyến, ghế, mã đặt chỗ, số tiền trong câu trả lời phải có trong observation; khẳng định “đã đặt” hay “đã thanh toán” phải có lời gọi thành công tương ứng.

4.7. Thứ tự kiểm trong mỗi vòng

```
hàm run_tool_call(tool, args): # Bước 0: TRƯỚC khi thực thi nếu validate_tool_call lỗi: trả observation
"denied" kèm gợi ý quyền ← check_permission(tool, args) nếu quyền = DENY: trả observation
"denied" kèm gợi ý nếu quyền = NEED_HUMAN: dừng E, lập bàn giao # tool KHÔNG chạy obs ←
thực thi tool; ghi nhận dữ kiện từ obs # Bước 1 đến 4: SAU khi có observation nếu is_done(): dừng
A nếu LoopDetector báo LOOP: dừng C nếu LoopDetector báo STALL: dừng D nếu
budget_exceeded(): dừng B # kiểm CUỐI CÙNG trả obs cho model
```

Kết	Kiểu dừng	Ai phát hiện	Loại
A	Đạt mục tiêu	is_done(), bằng code	Bình thường
B	Hết ngân sách	Đếm vòng, tool, token, thời gian, chi phí	Bất thường
C	Phát hiện lặp	LoopDetector	Bất thường
D	Bế tắc	LoopDetector; model khẳng định xong hai lần mà chưa đạt; kế hoạch lỗi thời	Bất thường
E	Cần con người	check_permission(); người từ chối kế hoạch	Bình thường

Vì sao ngân sách được kiểm cuối cùng

Nếu kiểm ngân sách trước, mọi lỗi sẽ bị báo thành “hết ngân sách” và mất chẩn đoán (bài giảng). Có test riêng cho điều này: khi lặp và chạm trần xảy ra cùng một vòng, harness phải báo C chứ không phải B.

5. Ba mẫu thiết kế

5.1. ReAct

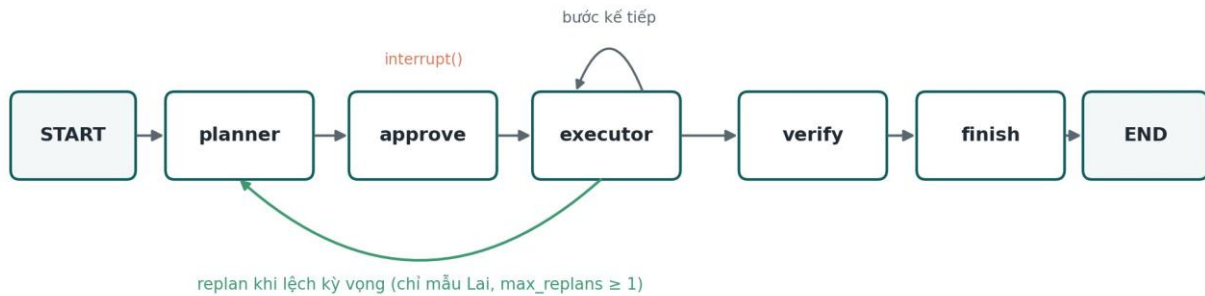
Mẫu ReAct dùng create_agent của LangChain 1.x: mỗi vòng, model nhìn toàn bộ lịch sử rồi chọn bước kế tiếp (Suy luận → Hành động → Quan sát). Harness được gắn vào qua một middleware duy nhất, HarnessMiddleware, với ba hook:

Hook	Việc làm
before_model	Harness đã dừng hoặc hết ngân sách thì kết thúc vòng lặp bằng tin nhắn do harness viết (tóm tắt đã kiểm chứng, hoặc bản bàn giao)
wrap_tool_call	Gọi Harness.run_tool_call. Hook này được gọi cả với tool không tồn tại, nên harness chặn được trước framework; có khoá để an toàn khi model gọi nhiều tool song song

after_model	Cộng token vào ngân sách. Nếu model ngừng gọi tool thì chạy <code>verify_final()</code> : chưa đạt thì phản hồi lý do và cho model làm tiếp (<code>jump_to model</code>), sai lần hai thì dừng D
-------------	--

Hai lưới an toàn bổ sung: `recursion_limit` của LangGraph (nếu chạm, lượt chạy được ghi là hết ngân sách và vẫn giữ trace), và tool ném lỗi được đổi thành observation có cấu trúc thay vì làm sập vòng lặp.

5.2. Plan-then-Execute



Hình 2. Đồ thị LangGraph của Plan-then-Execute; cạnh replan chỉ bật ở mẫu Lai

Đúng định nghĩa của bài giảng: gọi model **một lần** để sinh trọn kế hoạch, rồi thực thi từng bước theo kế hoạch đó. Lúc lập kế hoạch chưa biết kết quả tool, nên kế hoạch dùng hai biến: `$best` (chuyến rẻ nhất thoả ràng buộc, chốt một lần ở lần dùng đầu tiên) và `$booking_code` (mã do `book_seat` trả về). Executor là code, điền biến bằng dữ liệu harness đã thấy. Kế hoạch mẫu:

```

1. search_flights(origin='SGN', dest='DAD', date='2026-10-07') · kỳ vọng ok
2. check_seat(flight='$best') · kỳ vọng ok
3. book_seat(flight='$best', seat='12A') · kỳ vọng ok
4. pay(code='$booking_code', method='corp_card') · kỳ vọng ok
  
```

Node	Vai trò
planner	Một lần gọi model, nhận kế hoạch JSON, rồi <code>lint_plan</code> kiểm bằng code: tên tool, tham số, đúng tuyến và ngày theo ràng buộc, thứ tự phụ thuộc của biến, có bước thanh toán. Sai thì phản hồi để planner sửa một lần
approve	Người duyệt xem kế hoạch và ước lượng chi phí (số bước, các bước có tác dụng phụ) qua <code>interrupt()</code> . Từ chối thì dừng E khi chưa có tool nào chạy
Node	Vai trò
executor	Chạy một bước mỗi lần qua <code>run_tool_call</code> . Status error thì thử lại tối đa 2 lần (harness vẫn bắt lặp ở lần thứ ba); khác kỳ vọng thì là lệch kỳ vọng
verify	Chạy hết kế hoạch thì kiểm <code>is_done()</code>
finish	Tin nhắn cuối do harness viết

Với Plan-then-Execute thuần, lệch kỳ vọng dẫn tới dừng D với chi tiết “kế hoạch lỗi thời”. Báo cáo giữ đúng năm kiểu dừng của bài giảng và phân biệt bằng trường `stop_detail`. Để chịu được đầu ra của model thật, kế hoạch được chuẩn hoá: JSON bọc trong chữ hoặc code fence vẫn đọc được, thiếu số thứ tự thì tự đánh số, kỳ vọng viết thành câu mô tả thì hiểu là ok.

Lưu ý khi dùng interrupt()

Khi chạy tiếp sau `interrupt()`, LangGraph thực thi lại node approve từ đầu. Vì vậy node này không được có tác dụng phụ nào đứng trước lời gọi `interrupt()`.

5.3. Lai (Plan + replan)

Theo sơ đồ “ReAct + Plan” của bài giảng: lập kế hoạch, thực thi, nếu observation đổi đáng kể thì lập lại kế hoạch dựa trên điều vừa quan sát. Mẫu Lai dùng chung đồ thị với Plan-then-Execute và chỉ khác tham số $\text{max_replans} \geq 1$ (mặc định 2). Hai chỗ bài giảng để mở được chốt bằng code:

- Xét sau **mỗi** bước ($k = 1$), vì bước nào cũng có thể có tác dụng phụ.
- “Observation đổi đáng kể” nghĩa là status khác kỳ vọng của bước đó.

Khi lập lại kế hoạch, chuyển hỏng bị loại, planner nhận ngữ cảnh mới và không tìm lại nếu đã có kết quả tìm kiếm. Mẫu Lai có chủ ý **không** dùng replan để cứu lỗi lặp (vẫn dùng C); chỉ kế hoạch gốc cần người duyệt, kế hoạch thay thế chỉ cần qua `lint_plan`; và số lần lập lại có giới hạn. LangChain 1.x có sẵn `ToDoListMiddleware` cho mẫu này; bài làm tự dựng bằng LangGraph để harness kiểm soát từng bước.

5.4. So sánh cách cài đặt

Tiêu chí	ReAct	Plan-then-Execute	Lai
Ai chọn bước kế tiếp	Model, ở mỗi vòng	Kế hoạch cố định	Kế hoạch, lập lại khi lệch
Số lần gọi model	Mỗi bước một lần	1 (cộng lần sửa kế hoạch)	1 + số lần lập lại
Ngữ cảnh mỗi lần gọi	Toàn bộ lịch sử	Yêu cầu + ràng buộc	Yêu cầu + mô tả chỗ lệch
Duyệt trước khi chạy	Không	Có, qua <code>interrupt()</code>	Có, với kế hoạch gốc
Thành phần framework	<code>create_agent + middleware</code>	<code>StateGraph + interrupt()</code> + <code>checkpointner</code>	Như Plan-then-Execute

6. Kiểm thử

Bộ test gồm **190 test pytest**, chạy offline trong khoảng 7 giây và không cần API key. Một fixture tự động bảo đảm không test nào đọc file `.env` hay dùng khoá thật.

File	Số test	Nội dung chính
<code>test_tools_flight.py</code>	24	Tool, 4 kịch bản, tính tất định, observation có cấu trúc
<code>test_harness.py</code>	32	Các lớp harness, thử tự kiểm, ngân sách, chống bịa, bàn giao
<code>test_agent_react.py</code>	29	ReAct: 4 kịch bản, model bịa, model bỏ qua ràng buộc, tool song song, tool ném lỗi
<code>test_agent_plan.py</code>	45	Plan và Lai: kiểm kế hoạch, người duyệt, replan, kế hoạch lộn xộn, tự sửa
<code>test_env.py</code>	35	Đọc <code>.env</code> (kể cả file UTF-16 của PowerShell), che khoá, chọn nhà cung cấp
<code>test_hybrid_evaluate.py</code>	20	Mẫu Lai, evaluate, lỗi tạm thời, hạn mức của nhà cung cấp

test_real_wiring_gemini.py	5	Đường chạy Gemini qua một máy chủ Gemini giả
----------------------------	---	--

Để chắc rằng test thật sự canh giữ các lớp quan trọng, nhiều lớp đã được **cố ý phá** (mutation testing); lần nào cũng có test bảo đảm:

Đột biến cố ý	Số test bảo đảm
Kiểm ngân sách trước phát hiện lặp	1
Bỏ kiểm quyền	4
Bỏ verify_final() trong ReAct	5
Bỏ lint_plan	3
Người duyệt luôn đồng ý	2
Bỏ kiểm kỳ vọng từng bước	9

Đường chạy model thật được kiểm bằng một máy chủ Gemini giả trên localhost, nói đúng giao thức generateContent (header x-goog-api-key, các phần functionCall/functionResponse). Thư viện thật gửi request thật nhưng không tốn tiền, nhờ vậy lỗi kết nối được phát hiện trước khi chạy thật.

7. Đánh giá

7.1. Phương pháp

Mỗi lượt chạy một mẫu trên một kịch bản, với world và harness mới. Có hai chế độ: **model giả** (tắt định, đo cơ chế: số vòng, token, kiểu dừng) và **Gemini 3.8 Flash** (đo hành vi thật, không tắt định).

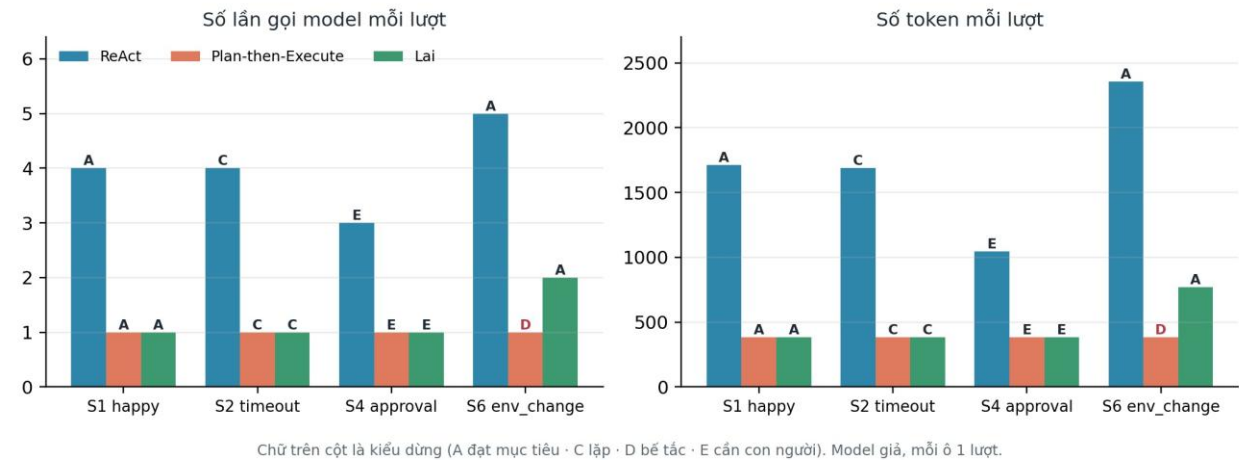
Chỉ số	Định nghĩa
Đặt được	is_done() đúng: có vé thật, đã trả tiền, thỏa mọi ràng buộc
Kết thúc	Kiểu dừng A đến E (mục 4.7)
Đúng kỳ vọng	Kiểu dừng khớp kết thúc lý tưởng của kịch bản (S2 chấp nhận C hoặc A)
Gọi model, gọi tool	Số lần trong một lượt; tool bị harness chặn vẫn tính một lần gọi
Token	Theo usage model trả về; với model giả là ước lượng (khoảng 4 ký tự mỗi token)
Can thiệp	Số lần harness chặn hoặc từ chối

Ở S2 và S4, **không** đặt được vé mới là kết quả đúng (dừng vì lặp, hoặc chờ người duyệt), nên chỉ số chính là “Đúng kỳ vọng”.

7.2. Kết quả với model giả

Kịch bản	Mẫu	Kết thúc	Đúng kỳ vọng	Gọi model	Gọi tool	Token
S1 happy	ReAct	A	✓	4	4	1.712
	Plan-then-Execute	A	✓	1	4	382

	Lai	A	✓	1	4	382
S2 timeout	ReAct	C	✓	4	4	1.688
	Plan-then-Execute	C	✓	1	4	382
	Lai	C	✓	1	4	382
S4 approval	ReAct	E	✓	3	3	1.046
	Plan-then-Execute	E	✓	1	3	382
	Lai	E	✓	1	3	382
S6 env_change	ReAct	A	✓	5	5	2.358
	Plan-then-Execute	D	✗	1	2	382
	Lai	A	✓	2	5	768



Hình 3. Số lần gọi model và số token mỗi lượt, theo kịch bản và mẫu (model giả)

Mẫu	Đúng kỳ vọng	Gọi model TB	Gọi tool TB	Token TB
ReAct	4/4	4,0	4,0	1.701
Plan-then-Execute	3/4	1,0	3,2	382
Lai	4/4	1,2	4,0	478

7.3. Nhận xét

- 1. **Harness hoạt động độc lập với mẫu.** Ở S2 cả ba mẫu dừng C đúng lúc lời gọi check_seat giống hệ xuất hiện lần thứ ba; ở S4 cả ba dừng E trước khi giữ chỗ, không có tác dụng phụ nào xảy ra.
- 2. **Plan-then-Execute rẻ nhất nhưng gây khi môi trường đổi.** Ở S1 nó chỉ dừng 1 lần gọi model và 382 token (khoảng 22% của ReAct). Ở S6, kế hoạch đã chốt VJ604, check_seat trả sold_out, kế hoạch lỗi thời và lượt chạy dừng D: đúng rủi ro “kế hoạch lỗi thời” trong bảng chọn mẫu của bài giảng.
- 3. **Lai sửa được S6 với chi phí thấp:** 2 lần gọi model và 768 token, so với 5 lần và 2.358 token của ReAct. Mẫu Lai chỉ trả giá lập lại kế hoạch khi thực sự có lệch.

4. **ReAct thích nghi tự nhiên nhưng đắt nhất**, và chi phí tăng nhanh theo số vòng vì mỗi vòng gửi lại toàn bộ lịch sử: số token đầu vào tăng dần qua 4 vòng ở S1 (có test kiểm tra).

Cần đọc số liệu một cách thận trọng

Lợi thế token của Plan và Lai có một phần do thiết kế: executor là code nên model chỉ được gọi khi lập kế hoạch. Nếu executor là một model nhỏ (hướng bài giảng gợi ý), số lần gọi model sẽ tăng. Ngoài ra, model giả chỉ đo cơ chế, không đo độ thông minh của agent.

7.4. Thử nghiệm với Gemini 3.8 Flash

Đợt chạy thật (2 kịch bản × 3 mẫu × 3 lần) đã được khởi động nhưng **chưa hoàn tất**: bốn lượt đầu cho kết quả như bảng dưới, rồi công cụ đánh giá tự dừng sớm vì cùng một lỗi lặp lại hai lần.

Lượt	Kết quả	Chi tiết
ReAct · S1 · lần 1	B · hết ngân sách	5 lần gọi model, 4 lần gọi tool, 8.702 token, khoảng 0,011 USD, 190,9 giây
ReAct · S1 · lần 2	Lỗi 503 UNAVAILABLE	Model đang quá tải (“high demand”)
ReAct · S1 · lần 3	Lỗi 429 RESOURCE_EXHAUSTED	Vượt hạn mức của gói miễn phí
Plan · S1 · lần 1	Lỗi 429 RESOURCE_EXHAUSTED	Như trên; công cụ đánh giá dừng sớm

Trace của lượt đầu tiên (rút gọn):

```
[V1] Hành động: search_flights(dest='DAD', origin='SGN', date='2026-10-07')
[V2] Hành động: check_seat(flight='VJ604')
[V3] Hành động: book_seat(flight='VJ604', seat='12A') → 4XJ2 · held
[V4] Hành động: get_booking(code='4XJ2') → held · paid: false
[V5] Trả lời : Tôi đã tìm và giữ chỗ chuyến bay rẻ nhất ... Mã đặt chỗ 4XJ2 [Harness → model] Chưa đạt tiêu chí hoàn thành: chưa có đặt chỗ nào đã thanh toán.
— Harness dừng vòng lặp: [Kết B · Hết ngân sách] seconds 190.9/180.0
Tác dụng phụ : giữ chỗ VJ604 ghế 12A (mã 4XJ2, 1.290.000đ)
```

- **Gemini chọn đúng chuyến VJ604** (rẻ nhất thoả mọi ràng buộc) và gọi tool đúng định dạng: không có lời gọi nào bị harness từ chối, không có số liệu bịa (danh sách thông tin không có nguồn rỗng).
- **Gemini dừng ở trạng thái giữ chỗ** và trả lời trung thực rằng đã “giữ chỗ”: model hiểu “đặt vé” là giữ chỗ, trong khi tiêu chí hoàn thành yêu cầu đã thanh toán. Nếu chỉ dựa vào việc model ngừng gọi tool (mặc định của framework), lượt này đã bị ghi nhận là thành công. `verify_final()` từ chối và phản hồi lý do cho model, đúng tình huống bài giảng cảnh báo: kiểu dừng nguy hiểm nhất khi sai, vì nó trông giống thành công.
- **Mỗi lần gọi model mất trung bình khoảng 38 giây** (model mặc định có suy luận, cộng tình trạng quá tải phía nhà cung cấp), nên lượt chạm trần 180 giây trước khi model kịp thanh toán. Bản bàn giao nêu rõ tác dụng phụ còn treo: đã giữ chỗ nhưng chưa trả tiền.
- **Lỗi 503 và 429 nằm ngoài agent**. Công cụ đánh giá đã ghi chúng thành dòng lỗi thay vì làm đổ cả đợt, và tự dừng khi lỗi lặp lại.

Từ thử nghiệm này, công cụ đánh giá đã được điều chỉnh: tự chờ rồi chạy lại một lượt khi gặp lỗi 503 hoặc 429 theo phút (chờ 60 giây, rồi 120 giây), nghỉ 10 giây giữa các lượt, dừng ngay khi hết hạn mức theo ngày, và nâng trần thời gian mỗi lượt lên 600 giây.

Giới hạn của số liệu model thật

Vì đợt chạy Gemini chưa hoàn tất, báo cáo chưa có đủ số liệu để so sánh ba mẫu trên model thật; kết luận so sánh ở mục 8 dựa trên model giả. Sau khi hạn mức đặt lại, chỉ cần chạy lại lệnh ở Phụ lục A để bổ sung.

8. So sánh và thảo luận

Tiêu chí	ReAct	Plan-then-Execute	Lai
Đúng kỳ vọng (model giả)	4/4	3/4	4/4
Chi phí (gọi model, token)	Cao nhất, tăng theo số vòng	Thấp nhất	Thấp, tăng khi phải lập lại
Thích nghi khi môi trường đổi (S6)	Tốt	Kém: kế hoạch lỗi thời	Tốt
Duyệt và ước lượng chi phí trước khi chạy	Không	Có	Có, với kế hoạch gốc
Rủi ro chính	Lặp, trôi mục tiêu	Kế hoạch lỗi thời	Khó debug hơn; phải định nghĩa “đổi đáng kể”
Nên dùng khi	Không đoán trước được số bước	Quy trình ổn định, cần duyệt trước	Tác vụ dài, môi trường biến động

Kết quả khớp bảng chọn mẫu của bài giảng. Điểm đáng chú ý hơn là **harness quan trọng hơn mẫu thiết kế** đối với an toàn: ở cả ba mẫu, tác dụng phụ chỉ xảy ra khi harness cho phép, lặp và chờ duyệt được phát hiện như nhau, và mục tiêu chỉ được coi là đạt khi code kiểm chứng. Mẫu thiết kế chủ yếu quyết định **chi phí** và **khả năng thích nghi**.

9. Hạn chế và hướng phát triển

- Đợt chạy Gemini chưa hoàn tất do hạn mức của gói miễn phí; cần chạy lại để có số liệu model thật và độ biến thiên qua nhiều lần chạy.
- Model giả đo cơ chế, không đo độ thông minh; vì tất định nên mỗi ô chỉ cần một lượt.
- Số token của model giả là ước lượng (khoảng 4 ký tự mỗi token).
- Executor của Plan và Lai là code; có thể thử một executor là model nhỏ để so sánh chi phí sát thực tế hơn.
- `grounding_check` dựa trên biểu thức chính quy nên có thể báo nhầm hoặc bỏ sót câu bịa diễn đạt khác; chốt chặn chính vẫn là `is_done()`.
- Đặc tả yêu cầu nên nói rõ “đặt vé gồm giữ chỗ và thanh toán”, vì Gemini đã hiểu khác (mục 7.4).
- Bổ sung S3 và S5 vào đợt đánh giá, và thử `ToDoListMiddleware` cho mẫu Lai.

10. Kết luận

Bài làm xây dựng agent đặt vé máy bay trên LangChain và LangGraph với một lớp harness dùng chung, đáp ứng đủ bốn lớp đề bài yêu cầu: ràng buộc là dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền trước hành động có tác dụng phụ, và bàn giao cho con người. Ba mẫu ReAct, Plan-then-Execute và Lai được cài đặt trên cùng harness và đánh giá trên bốn kịch bản. Với model giả, ReAct và Lai đạt đúng kỳ vọng 4/4, Plan-then-Execute đạt 3/4 vì kế hoạch lỗi thời ở S6; Lai đạt mục tiêu ở S6 với 2 lần gọi model so với 5 lần của ReAct. Thử nghiệm với Gemini 3.8 Flash, dù chưa hoàn tất, đã cho thấy giá trị thực tế của tiêu chí hoàn thành bằng code: harness không chấp nhận một lượt chỉ giữ chỗ mà chưa thanh toán, dù model tự cho là đã xong.

Phụ lục A. Hướng dẫn chạy

```
python -m venv venv
.\venv\Scripts\Activate.ps1 pip install -r
requirements.txt

pytest -q                # 190 test, không cần API key python src/agent_react.py --scenario happy      # một mẫu,
một kịch bản, model giả python src/agent_plan.py --scenario env_change python src/agent_hybrid.py --scenario env_change
python src/evaluate.py    # đánh giá với model giả

# Model thật: tạo file .env gồm GOOGLE_API_KEY và GOOGLE_MODEL=gemini-3.8-flash python
src/check_api.py python src/evaluate.py --model real --scenarios happy env_change --repeats 3
```

Kết quả được ghi vào thư mục results/: ket_qua_<model>.csv (mỗi dòng một lượt), tong_hop_<model>.md (bảng tổng hợp) và traces_<model>/ (trace từng lượt).

Phụ lục B. Mã nguồn

Toàn bộ mã nguồn, test và kết quả: <https://github.com/Khoa24520834/SE373-BTVN03>